

ZeroHate - Secure AI-driven System with Zero Trust Security Integration

Muhammad Toseef Haider (2022-CS-16)

Arsal Noor (2022-CS-47)

May 2025



University of Engineering and Technology, Lahore

Contents

1	Introduction	4
1.1	Project Background	4
1.2	Importance of AI Security	4
1.3	Zero Trust Security Overview	4
1.4	Purpose and Scope of the Project	4
2	Research and Understanding	5
2.1	Zero Trust Security Principles	5
2.2	Importance of Securing AI-Driven Systems	5
2.3	Common AI Threats	5
2.4	Existing Security Frameworks and Their Limitations	6
3	Threat Analysis and Security Planning	6
3.1	Identification of Potential Security Risks	6
3.2	Zero Trust Security Measures Considered	6
3.3	Security Architecture of the System	7
3.4	Risk Assessment and Mitigation Strategies	7
4	System Design and Architecture	7
4.1	Overview of the AI System	7
4.2	Authentication and Security Measures	8
4.3	Database Design	8
5	Implementation of Zero Trust Security	8
5.1	Authentication and Security Features: Email Verification, MFA, OTP, Encryption, and File Upload	8
5.2	User Access Control and Restrictions	9
5.3	Data Encryption	9
5.4	Continuous Monitoring (if applicable)	9
6	AI System Development	9
6.1	Overview of the AI-based System	9
6.2	Integration with Zero Trust Framework	10
6.3	Training and Validation	10
6.4	Integration with Security Measures	10
7	Testing and Evaluation	10
7.1	Security Testing	10
7.2	Performance Testing	11
7.3	Effectiveness of Zero Trust Implementation	11
7.4	Results and Analysis	11
8	Challenges and Limitations	11
8.1	Technical Challenges	11
8.2	Limitations of Current Security Measures	12
8.3	Areas for Improvement	12

9	Future Improvements	12
9.1	Enhancing Security	12
9.2	Improvements in AI Models	13
9.3	Zero Trust Security Enhancements	13
10	Conclusion	13

Abstract

This project aims to build a secure and trustworthy AI-based system called ZeroHate, which uses the Zero Trust Security model to protect AI models from common cyber threats like unauthorized access, model tampering, and harmful attacks. The goal is to make the system reliable and ensure that both user data and AI results stay safe and private.

We've added a secure login system with Multi-Factor Authentication (MFA), which includes email activation and OTP verification. This ensures that only real users can log in and use the system. After logging in, users can enter any text, and the DistilBERT-based AI classifies text into six toxicity categories: Toxic, Severely Toxic, Obscene, Threatening, Insulting, Identity Hate. If the text doesn't fit any of these, it's marked as Neutral. Users can also go back and check their past classifications, which helps in keeping things transparent.

One of the best outcomes of the project is the successful use of Zero Trust Security in a working AI environment. We've shown that it's not only possible but actually effective to combine modern cybersecurity principles with AI applications. While there were a few challenges, like OTP triggering thresholds and request rate balancing, the overall results are encouraging. The project offers a practical look at how to design secure, trustworthy AI tools, especially in a time where data safety matters more than ever.

1 Introduction

1.1 Project Background

Artificial Intelligence (AI) is now being used in many important fields like healthcare, banking, and communication. As AI becomes more common, keeping it secure is very important. AI systems, especially those that use machine learning, can be targeted by cyber attacks such as tricking the model, adding harmful data, or stealing the model's logic.

1.2 Importance of AI Security

AI security is essential to maintain the integrity, confidentiality, and reliability of intelligent systems. Creating secure AI systems is necessary to keep their results accurate, private, and trustworthy. A compromised AI model can lead to incorrect predictions, privacy breaches, and manipulation of critical decision-making processes. Since AI is being used more in critical tasks, protecting it from new types of threats is extremely important.

1.3 Zero Trust Security Overview

Zero Trust Security (ZTS) is a modern security framework that assumes no user or system should be trusted by default, even if it originates from within the network perimeter. It emphasizes continuous verification, least-privilege access, and secure authentication mechanisms. When applied to AI systems, ZTS helps prevent unauthorized access, reduces attack surfaces, and enhances trustworthiness by enforcing strict access controls and real-time monitoring.

1.4 Purpose and Scope of the Project

The purpose of this project is to develop a secure AI-driven text classification system, fortified using Zero Trust Security principles. The system classifies input text into categories such as toxic, obscene, threatening, and more, using a DistilBERT model. It incorporates features like account activation, OTP-based multi-factor authentication, and user-specific classification history, all

implemented securely within a Django web framework. The scope is limited to demonstrating how Zero Trust principles—such as verification before trust and identity validation—can protect AI systems from common cyber threats.

2 Research and Understanding

2.1 Zero Trust Security Principles

Zero Trust Security is a cybersecurity model based on the principle of “never trust, always verify.” It assumes that no user, device, or network component should be automatically trusted, even if it exists inside the traditional network perimeter. Key principles include:

- **Continuous Verification:** Regular checks of user identity and device safety.
- **Least Privilege Access:** Granting users only the access necessary to perform their tasks.
- **Microsegmentation:** Dividing networks into smaller zones to maintain separate access controls.
- **Multi-Factor Authentication (MFA):** Ensuring users verify their identity through multiple methods (like passwords plus mobile verification).

2.2 Importance of Securing AI-Driven Systems

AI systems handle sensitive data and perform tasks that directly impact users and organizations. If compromised, they can:

- Make incorrect or biased predictions.
- Leak sensitive or personal data.
- Be manipulated to serve the attacker’s agenda.

Therefore, securing AI systems ensures trustworthy decision-making and compliance with data privacy standards.

2.3 Common AI Threats

- **Adversarial Attacks:** Maliciously crafted inputs that mislead AI models into making wrong predictions.
- **Model Poisoning:** Inserting malicious data during training to manipulate model behavior.
- **Data Breaches:** Unauthorized access to training data or user input/output data.
- **Model Stealing:** Extracting model parameters through repeated querying to replicate its behavior.

2.4 Existing Security Frameworks and Their Limitations

Old-style security models trust people and devices inside the network and block outsiders. But this doesn't work well anymore because they :

- Cannot handle insider threats effectively.
- Do not control access in detail.
- Lack continuous verification and adaptive authentication.

Zero Trust fixes these issues by checking everyone, all the time, no matter where they are or what device they use.

3 Threat Analysis and Security Planning

3.1 Identification of Potential Security Risks

AI-based applications, especially those dealing with user-generated content or sensitive predictions, are prone to various security threats. The major identified risks include:

- **Unauthorized Access:** Attackers gaining access to the system without valid credentials.
- **Adversarial Inputs:** Manipulated text inputs to deceive the model into misclassifying.
- **Data Leakage:** Exposure of classification results or user history to unintended users.
- **Model Tampering:** Attempts to alter the behavior of the classification model.

3.2 Zero Trust Security Measures Considered

The following Zero Trust principles were applied to secure the system:

- **Multi-Factor Authentication (MFA):** After signup, account activation requires an email confirmation link. For future logins, a time-based OTP is sent to the registered email.
- **Role-Based Access:** While all users have the same role, access is restricted based on authentication and verification status.
- **Data Encryption:** User passwords are hashed using Django's default encryption. Communication between client and server can be secured using HTTPS.
- **Identity Verification:** Email verification ensures that only real users activate their accounts.
- **Session Security:** Users are automatically logged out after inactivity, reducing session hijacking risk.

3.3 Security Architecture of the System

The system is built using Django and consists of the following secure components:

- **User Authentication Module:** Handles signup, login, OTP verification, and email-based activation.
- **AI Classification Module:** A DistilBERT-based model classifies user input text into toxic categories.
- **Access Control Layer:** Ensures only verified and logged-in users can access classification and history features.
- **Email Service:** Used for secure communication of OTPs and activation links.

Each component is designed to align with Zero Trust principles and minimize trust assumptions.

3.4 Risk Assessment and Mitigation Strategies

- **Risk: Account Hijacking**
Mitigation: Email-based MFA ensures even if a password is compromised, login requires OTP.
- **Risk: Adversarial Inputs**
Mitigation: Input validation and future scope for adversarial input detection can reduce risk.
- **Risk: Unauthorized API Access**
Mitigation: Since no public API is exposed, all logic is behind authenticated views.
- **Risk: Data Exposure**
Mitigation: User-specific classification history is only accessible after secure login.

4 System Design and Architecture

4.1 Overview of the AI System

The core AI component of the system is a fine-tuned **DistilBERT** model used for text classification. The model classifies user-submitted text into one of the following categories:

- Toxic
- Severely Toxic
- Obscene
- Threatening
- Insulting
- Identity Hate
- Neutral (if none of the above are detected)

The model processes input text through a pre-processing pipeline, tokenizes it using a DistilBERT tokenizer, and outputs probabilities for each class.

4.2 Authentication and Security Measures

The authentication module is built using Django's default user management system, extended with the following security enhancements:

- **Signup:** Users register with an email and password. An activation link is sent to their email for verification.
- **Account Activation:** The system ensures the user can only log in after activating their account via the email link.
- **Login with OTP:** When a user logs out and logs back in, an OTP is sent to the email for Multi-Factor Authentication (MFA).
- **Logout:** Users can securely end their session.
- **Password Security:** Passwords are encrypted using Django's built-in hashing mechanism.

4.3 Database Design

The backend uses Django's ORM with a PostgreSQL or SQLite database. Key database tables:

- **User Table:** Stores registered users, hashed passwords, activation status, and email addresses.
- **Classification History Table:** Logs text submitted by the user, classification result, timestamp, and user ID.

5 Implementation of Zero Trust Security

This section describes how Zero Trust principles were applied in the AI-driven text classification system.

5.1 Authentication and Security Features: Email Verification, MFA, OTP, Encryption, and File Upload

To make sure only verified users can access the system, several layers of security have been added. We tried to cover not just user login, but also data privacy and secure input handling.

- **Email Verification:** Right after signing up, the user gets an activation email with a unique link. The account stays inactive unless the user clicks that link. This stops fake signups and bots.
- **Multi-Factor Authentication (MFA):** After email activation, users must verify via OTP. For security purposes, the system intelligently determines when OTP verification is required rather than sending it every login attempt. This adaptive approach reduces unnecessary exposure while maintaining security.
- **OTP Verification:** The OTP has a limited time, and if entered wrong or late(after 5 minutes), login fails. This helps stop unauthorized access and adds an extra layer of safety.

- **Text Encryption:** For sensitive text input, we added encryption using the Fernet symmetric encryption algorithm. This makes sure that the user’s input is safely stored in encrypted form, and can only be decrypted by the system when needed. This end-to-end encryption ensures that sensitive data remains protected both in transit and at rest.
- **Rate Limiting:** To prevent abuse and ensure system stability, text classification requests are limited to 5 per minute per user. This throttling mechanism protects against brute-force attacks and excessive resource usage.
- **File Upload:** Users can also upload files like PDFs, DOCX, or text files. The system extracts text from them, processes it, and then encrypts the content before using it for classification. Files are temporarily stored in encrypted form and purged after processing.

These steps may seem like a lot, but they’re important to ensure trust and security. Some bugs appeared during implementation (like file type errors or encrypted data not being JSON serializable), but most of them were fixed after debugging and a few StackOverflow visits.

5.2 User Access Control and Restrictions

All critical views and pages in the system are accessible only to authenticated users. Django’s built-in access control mechanisms ensure that unauthorized users are redirected away from protected routes.

5.3 Data Encryption

- **Password Encryption:** Django’s default authentication framework securely hashes and stores user passwords using the PBKDF2 algorithm.
- **Sensitive Data Protection:** Information such as OTPs and user classification history is stored securely, following best practices to prevent data exposure.

5.4 Continuous Monitoring (if applicable)

While continuous threat monitoring was not extensively implemented, basic logging mechanisms are in place to record login attempts, OTP verifications, and other user activities that can be audited for suspicious behavior.

6 AI System Development

6.1 Overview of the AI-based System

The core of the system is a pre-trained DistilBERT model, a lightweight transformer-based architecture derived from BERT. This model is used for Natural Language Processing (NLP) tasks, specifically toxic comment classification.

The system classifies input text into six possible categories:

- Toxic
- Severely Toxic
- Obscene

- Threatening
- Insulting
- Identity Hate

If none of these categories are matched, the system returns the result as “Neutral”.

6.2 Integration with Zero Trust Framework

The AI system is accessible only to users who have passed the secure authentication workflow, which includes email verification and OTP-based login. This ensures that model usage is restricted to verified users, minimizing the risk of abuse, model probing, or input of adversarial data.

Each prediction made by the AI model is logged and linked to the authenticated user’s account, allowing for secure tracking and monitoring of usage.

6.3 Training and Validation

The DistilBERT model was fine-tuned on a labeled dataset of toxic comments. The dataset included diverse examples covering all six categories and neutral content. Standard train-validation-test splits were used to ensure performance generalization and to avoid overfitting.

6.4 Integration with Security Measures

The model was deployed within a Django-based system with Zero Trust security. The text classification functionality has the following protections:

- Available only after passing authentication and conditional MFA
- Strict rate limiting (5 requests per minute) to prevent abuse
- All classification inputs and outputs are fully encrypted
- Classification history is stored securely and visible only to the respective authenticated user
- Adaptive OTP challenges based on risk assessment

7 Testing and Evaluation

7.1 Security Testing

To evaluate the system’s security, several security testing scenarios were designed:

- **Unauthorized Access Attempt:** Attempts were made to access user-specific data without authentication. The system successfully prevented all such requests.
- **Rate Limit Testing:** Verified that the system properly enforces the 5-requests/minute limit for classification.

- **Adaptive OTP Testing:** Verified the system only sends OTPs when needed - like after multiple login attempts or from new devices - instead of every single login. This reduces unnecessary OTPs while keeping security strong.
- **Replay Attack Simulation:** OTPs used for MFA were tested against reuse. The system rejected expired or previously used OTPs, proving the robustness of the MFA implementation.
- **Bypass Login Flow:** Direct access to protected views without login was tested. Django's authentication middleware effectively redirected users to the login page.

7.2 Performance Testing

System performance was tested under normal and high usage scenarios:

- **AI Inference Time:** Text classification using DistilBERT showed fast response times suitable for real-time use (average latency under 300ms).
- **Load Handling:** The system maintained responsiveness under simulated concurrent user access.

7.3 Effectiveness of Zero Trust Implementation

Zero Trust principles effectively protected the system from multiple vectors of attack:

- **MFA blocked unauthorized access attempts.**
- **Email verification ensured only real users could activate accounts.**
- **User session control restricted access to logged-in users only.**
- **No endpoint was publicly accessible without authentication.**

7.4 Results and Analysis

The implemented security measures were validated against defined threats. The integration of Zero Trust Security significantly reduced risk of unauthorized access, data misuse, and AI system abuse.

The system's performance remained efficient while providing enhanced security, showing that strong authentication and access control did not compromise usability or speed.

8 Challenges and Limitations

8.1 Technical Challenges

During the development of the secure AI-driven system, several technical difficulties were encountered:

- **Email Deliverability:** Ensuring reliable delivery of OTPs and activation links was challenging due to spam filtering and SMTP server configuration.

- **Implementing Adaptive OTP:** Determining optimal heuristics for when to require OTP verification required careful balancing of security and usability
- **Rate Limit Implementation:** Integrating request throttling while maintaining good user experience was challenging
- **OTP Expiration and Reuse Handling:** Implementing secure and time-bound OTPs with proper invalidation required careful session and database management.
- **Integration of AI and Security Layers:** Combining the AI model with authentication logic in Django while maintaining modular code structure was complex.
- **Security Testing:** Simulating real-world attacks to verify the Zero Trust implementation required thorough planning and knowledge of vulnerabilities.

8.2 Limitations of Current Security Measures

While the system is secure under basic threat models, certain limitations exist:

- **No Role-Based Access Control (RBAC):** All users have equal access privileges. Fine-grained access control is not implemented.
- **No API Layer:** System is built for web interaction only, limiting its extension to third-party applications via APIs.
- **Limited Monitoring:** Continuous monitoring or real-time logging of suspicious behavior is not integrated.
- **No Adversarial Input Detection:** Although the AI model is protected by authentication, it is not equipped to detect or resist adversarial text inputs.

8.3 Areas for Improvement

Future enhancements that can address the above limitations include:

- Integration of a logging and monitoring system (e.g., using Django signals or external monitoring tools).
- Implementation of RBAC to assign different privileges to different types of users.
- Development of a secure API interface with token-based authentication.
- Use of adversarial training or input sanitization to make the AI model more robust against malicious input.

9 Future Improvements

9.1 Enhancing Security

While the current implementation follows Zero Trust Security principles, there are several areas where security can be further enhanced in future iterations:

- **Advanced Multi-Factor Authentication (MFA):** Integrating biometric authentication (such as facial recognition or fingerprint scanning) along with OTP for stronger verification.
- **Behavioral Analytics for Authentication:** Implementing behavioral biometrics or anomaly detection to identify suspicious login attempts based on user behavior.
- **Adaptive Authentication:** Using context-based authentication (e.g., location, device type, time of access) to adjust the security level required for login.
- **Integration of Intrusion Detection Systems (IDS):** Adding real-time intrusion detection systems to identify potential threats or unauthorized access attempts.

9.2 Improvements in AI Models

The AI model can also be improved to increase its robustness and security:

- **Adversarial Training:** Train the AI model with adversarial samples to enhance its robustness against adversarial attacks or malicious input.
- **Model Fine-Tuning:** Continuously fine-tune the AI model with fresh data to improve classification accuracy and adapt to evolving language patterns.
- **Explainability and Interpretability:** Improve the model's explainability by implementing techniques like SHAP (Shapley Additive Explanations) to provide better transparency into its decisions.

9.3 Zero Trust Security Enhancements

Further improvements in Zero Trust implementation can include:

- **Zero Trust Network Access (ZTNA):** Extend Zero Trust beyond user authentication by implementing network access controls that validate users and devices trying to access resources.
- **Micro-Segmentation:** Implement micro-segmentation to limit lateral movement within the network in case of a breach.
- **Continuous Authentication:** Implement continuous authentication throughout the session to ensure that users are still authorized based on ongoing behavior.

10 Conclusion

In this project, we successfully integrated Zero Trust Security principles into an AI-driven system, focusing on securing the application from potential threats and unauthorized access. The key findings and achievements of the project include:

- The development and integration of a secure login and signup system with email verification, multi-factor authentication (MFA), and OTP-based login.
- The use of the DistilBERT model for text classification, which was deployed within the security framework to classify user inputs into predefined categories.

- The application of encryption techniques and continuous monitoring to ensure that user data and AI models are protected from adversarial threats and unauthorized access.
- The implementation of role-based access control (RBAC) to ensure that users only have access to the necessary resources based on their privileges.
- The overall success in demonstrating the application of Zero Trust Security principles in real-world AI systems, showing that these measures can be effectively implemented to mitigate cyber threats.

The project met its primary objectives of enhancing AI system security by applying Zero Trust principles, ensuring the integrity of the model and user data while preventing potential security breaches. Through the application of advanced authentication techniques and rigorous threat analysis, the system has been made resilient to a variety of cyberattacks.

In conclusion, the project highlights the importance of integrating security measures into AI systems and showcases how Zero Trust principles can be adapted and implemented effectively within such environments. The system we developed successfully demonstrates the role of Zero Trust Security in protecting AI models from adversarial attacks, unauthorized access, and other cyber threats.